

What Is System Design (HLD vs LLD)

Phase 1 · Fundamentals · Estimated time: 1 hour

1. Concept From First Principles

English:

You already design systems every day — you just haven't called it that. When you build a Laravel application, you decide things like: should this run as a background job or inline? Should we cache this expensive query in Redis? Should file uploads go straight to disk or to S3? Every one of those decisions is a small act of system design. High-Level Design (HLD) is the same kind of thinking, done deliberately, at the scale of an entire product, before a single line of code is written.

Hindi:

आप हर दिन systems design करते ही हैं — बस आपने इसे कभी इस नाम से नहीं जाना। जब आप एक Laravel application बनाते हैं, तो आप ऐसी चीज़ें decide करते हैं: क्या यह background job की तरह चलेगा या inline? क्या हमें इस expensive query को Redis में cache करना चाहिए? क्या file uploads सीधे disk पर जाएँ या S3 पर? यह हर decision, system design का एक छोटा सा हिस्सा है। High-Level Design (HLD) भी बिल्कुल यही सोच है, बस इसे जानबूझकर, पूरे product के स्तर पर, कोड लिखने से पहले किया जाता है।

Important Words:

- **Background job** – बैकग्राउंड जॉब
- **Inline** – इनलाइन / सीधे
- **Cache** – कैश करना
- **Deliberately** – जानबूझकर

English:

Formally: System design is the process of defining the architecture, components, data flow, and interfaces of a system to satisfy specified functional and non-functional requirements (scale, availability, latency, cost). It sits above code — it answers what pieces exist and how they talk to each other, not what does each function do internally.

Hindi:

औपचारिक रूप से: System design वह process है जिसमें किसी system की architecture, components, data flow, और interfaces को इस तरह define किया जाता है कि वे दिए गए functional और non-functional requirements (scale, availability, latency, cost) को पूरा करें। यह code से ऊपर की चीज़ है — यह बताता है कि कौन-कौन से पीस मौजूद हैं और वे आपस में कैसे बात करते हैं, ना कि हर function अंदर क्या करता है।

Important Words:

- **Architecture** – आर्किटेक्चर / ढांचा
- **Component** – कॉम्पोनेंट / हिस्सा
- **Non-functional requirements** – नॉन-फंक्शनल रिक्वायरमेंट्स
- **Availability** – उपलब्धता
- **Latency** – लेटेसी / देरी

HLD vs LLD — the actual difference

English:

High-Level Design (HLD) is the architecture blueprint: which services exist, which databases they use, how they communicate (REST? a queue?), where the bottlenecks will be, and how the system behaves under load or failure. Deliverables look like box-and-arrow diagrams, API contracts, and capacity estimates.

Hindi:

High-Level Design (HLD) एक architecture blueprint है: कौन-कौन सी services मौजूद हैं, वे कौन सा database use करती हैं, वे आपस में कैसे communicate करती हैं (REST? या queue?), bottleneck कहाँ होंगे, और load या failure के समय system कैसा behave करता है। इसके deliverables box-and-arrow diagrams, API contracts, और capacity estimates जैसे होते हैं।

Important Words:

- **Blueprint** – खाका / योजना
- **Bottleneck** – बॉटलनेक / रुकावट
- **Deliverable** – डिलीवरेबल / परिणाम
- **Capacity estimate** – क्षमता अनुमान

English:

Low-Level Design (LLD) zooms into a single box from the HLD diagram and asks: what classes, functions, design patterns, and database tables actually implement this? If you were designing a parking lot system, the HLD says "there is a Booking Service backed by a relational database"; the LLD says "here is the ParkingSpot class, the Vehicle class, and the strategy pattern used to price a ticket."

Hindi:

Low-Level Design (LLD) HLD diagram के किसी एक box के अंदर zoom करके पूछता है: इसे actually implement करने के लिए कौन-कौन सी classes, functions, design patterns, और database tables चाहिए? अगर आप एक parking lot system design कर रहे हैं, तो HLD कहता है "एक Booking Service है जो relational database पर based है"; जबकि LLD कहता है "यह रहा ParkingSpot class, यह Vehicle class, और यह strategy pattern जो ticket की price निकालने के लिए use होता है।"

Important Words:

- **Zoom into** – अंदर जाकर देखना
- **Design pattern** – डिज़ाइन पैटर्न
- **Relational database** – रिलेशनल डेटाबेस

Backend engineer analogy

English:

Think of HLD as the decision to use a queue (RabbitMQ/SQS) so your Laravel app doesn't block on sending emails synchronously. LLD is then writing the actual Job class, its handle() method, and retry logic. You've been doing LLD-adjacent work for 9 years. HLD asks you to zoom out one level further and reason about the whole system, often for a product you've never built, in 30-45 minutes, out loud.

Hindi:

HLD को ऐसे समझें: यह वह decision है कि emails synchronously भेजने की बजाय एक queue (RabbitMQ/SQS) use किया जाए ताकि आपका Laravel app block ना हो। LLD फिर वह असली Job class लिखना है, उसका handle() method, और retry logic। आप पछिले 9 साल से LLD जैसा ही काम करते आए हैं। HLD आपसे कहता है कि एक level और ऊपर जाकर पूरे system के बारे में सोचें, अक्सर ऐसे product के लिए जो आपने कभी बनाया ही नहीं, वो भी 30-45 मिनट में, जोर से बोलकर।

Important Words:

- **Synchronously** – सक्रोनसली / एक साथ क्रम में

English:

A third, related term you'll hear: coding interviews test whether you can write correct, efficient code for a well-defined problem (e.g., "reverse a linked list"). HLD interviews test whether you can take an ambiguous, large problem ("design WhatsApp") and produce a workable architecture while explaining trade-offs. These are different muscles, and most 9-year backend engineers have only trained the first one.

Hindi:

एक तीसरा शब्द भी आप सुनेंगे: coding interviews यह test करते हैं कि क्या आप किसी well-defined problem (जैसे "reverse a linked list") के लिए सही और efficient code लिख सकते हैं। HLD interviews यह test करते हैं कि क्या आप एक ambiguous, बड़े problem ("design WhatsApp") को लेकर एक काम करने लायक architecture बना सकते हैं और साथ में trade-offs भी explain कर सकते हैं। यह दोनों अलग-अलग skills हैं, और ज्यादातर 9 साल के experience वाले backend engineers ने सिर्फ पहली वाली skill ही practice की होती है।

Important Words:

- **Ambiguous** – अस्पष्ट / अनश्चिति
- **Workable** – व्यावहारिक / काम करने लायक
- **Muscle (figurative)** – क्षमता / skill

English:

Here's the mistake almost every strong coder makes on their first HLD attempt: they start designing the database schema or writing pseudo-code within the first two minutes, because that's the muscle memory from years of feature work. An HLD interview rewards the opposite instinct — stay at the architecture altitude deliberately, resist the pull toward implementation detail, and only zoom in when the interviewer asks you to or when a specific component genuinely needs justification.

Hindi:

लगभग हर अच्छे coder की पहली HLD कोशिश में यही गलती होती है: वे शुरुआती दो मिनट में ही database schema design करने लगते हैं या pseudo-code लिखने लगते हैं, क्योंकि सालों के feature-work से यही आदत बन जाती है।

HLD interview इसके उल्टा instinct चाहता है — जानबूझकर architecture वाले level पर रुके, implementation detail में जाने की इच्छा को रोके, और तभी अंदर zoom करे जब interviewer कहे या जब किसी एक component को genuinely justify करने की ज़रूरत हो।

Important Words:

- **Pseudo-code** – स्यूडो-कोड
- **Instinct** – सहज प्रवृत्ति
- **Justification** – औचित्य / कारण देना

English:

It also helps to understand why companies weight HLD so heavily for backend roles specifically. A backend engineer with 9 years of experience is expected to eventually own architecture decisions, not just implement tickets — things like "should this new feature be its own service," or "will this design still work at 10x our current user base." HLD interviews are a proxy for exactly that judgment.

Hindi:

यह समझना भी फायदेमंद है कि companies backend roles के लिए HLD को इतना ज़्यादा महत्व क्यों देती हैं। 9 साल के experience वाले backend engineer से उम्मीद की जाती है कि वह आगे चलकर architecture decisions खुद ले सके, सिर्फ tickets implement ना करे — जैसे "क्या यह नया feature अपनी अलग service होनी चाहिए," या "क्या यह design हमारे मौजूदा users से 10 गुना ज़्यादा पर भी काम करेगा।" HLD interviews असल में इसी judgment को test करने का एक तरीका है।

Important Words:

- **Own (verb, figurative)** – ज़िम्मेदारी लेना
- **Proxy** – प्रॉक्सी / प्रतिनिधिभाष

2. Real-World Example / Case Study

English:

Consider Swiggy's order-placing flow. At the HLD level, the architecture is described as a set of cooperating services: a Restaurant Service (menu, availability), an Order Service (cart, checkout), a Payment Service (talks to a payment gateway), a Delivery-Partner Tracking Service (live location), and a Notification Service (SMS/push). The HLD conversation is about how these services communicate.

Hindi:

Swiggy के order-placing flow को देखें। HLD level पर, architecture को कई cooperating services के रूप में बताया जाता है: एक Restaurant Service (menu, availability), एक Order Service (cart, checkout), एक Payment Service (payment gateway से बात करती है), एक Delivery-Partner Tracking Service (live location), और एक Notification Service (SMS/push)। HLD की बातचीत इसी पर होती है कि ये services आपस में कैसे communicate करती हैं।

Important Words:

- **Cooperating services** – साथ मलिकर काम करने वाली services
- **Checkout** – चेकआउट
- **Payment gateway** – पेमेंट गेटवे

English:

Zoom into just the Order Service and you're now in LLD territory: an Order class with a state machine (CREATED -> CONFIRMED -> PREPARING -> OUT_FOR_DELIVERY -> DELIVERED), a repository

pattern for persistence, and validation logic for cart items.

Hindi:

सर्कि Order Service के अंदर zoom करें तो अब आप LLD की दुनिया में हैं: एक Order class जिसमें एक state machine है (CREATED -> CONFIRMED -> PREPARING -> OUT_FOR_DELIVERY -> DELIVERED), persistence के लिए एक repository pattern, और cart items के लिए validation logic।

Important Words:

- **State machine** – स्टेट मशीन
- **Repository pattern** – रपोजिटरी पैटर्न
- **Persistence** – परसिस्टेंस / स्थायी भंडारण
- **Validation** – वैलिडेशन / जांच

English:

This distinction is exactly why interview processes at Flipkart, PhonePe, Razorpay, Careem, and Noon-tier companies run separate HLD and LLD rounds for senior backend roles — they are testing different skills, and this program focuses entirely on the HLD skill.

Hindi:

यही वजह है कि Flipkart, PhonePe, Razorpay, Careem, और Noon जैसी companies senior backend roles के लिए HLD और LLD के अलग-अलग rounds रखती हैं — क्योंकि वे अलग-अलग skills test करती हैं, और यह पूरा program सर्कि HLD skill पर focused है।

English:

It's worth noting how this plays out slightly differently across the regions you're preparing for. Indian product companies commonly run a dedicated HLD round for SDE-2 and above. Gulf-based product and fintech teams frequently blend a shorter HLD discussion into a broader "system thinking" round, sometimes paired with a walkthrough of a system you've actually built.

Hindi:

यह जानना भी ज़रूरी है कि जिन regions के लिए आप तैयारी कर रहे हैं, वहाँ यह थोड़ा अलग तरीके से होता है। Indian product companies अक्सर SDE-2 और उससे ऊपर के लिए एक अलग HLD round रखती हैं। Gulf की product और fintech teams अक्सर एक छोटी HLD discussion को एक बड़े "system thinking" round में मिला देती हैं, कभी-कभी साथ में उस system की walkthrough भी होती है जो आपने खुद पहले बनाया हो।

Important Words:

- **Walkthrough** – वॉकथ्रू / वसितार से समझाना

3. Diagram (English labels kept as-is)

HLD view of a checkout flow (the level of detail an HLD interview expects):

```

[ Client App ]
  |
  v
[ API Gateway ]
  |
  +-----> [ Order Service ] --> [ Order DB ]
  |
  |               |
  |               +--> [ Message Queue ] --> [ Notification Serv
ice ]
  |
  +-----> [ Payment Service ] --> [ Payment Gateway (external)
]
  |
  +-----> [ Inventory Service ] --> [ Inventory DB / Cache ]

```

HLD stops here: boxes, data stores, and the arrows between them.

LLD view zooms into just the "Order Service" box:

```

class Order {
    id, userId, items[], status, totalAmount
    transitionTo(newStatus) // enforces valid state transitions
}

class OrderRepository {
    save(order)
    findById(id)
}

class OrderService {
    createOrder(cart) -> Order
    cancelOrder(id)
}

LLD goes here: class fields, methods, patterns, DB schema detail.

```

4. Trade-offs Table (Reference Table – English)

Aspect	HLD	LLD
Primary question	What components exist and how do they interact?	How is each component implemented internally?
Typical deliverable	Architecture diagram, API contracts, capacity plan	Class diagrams, method signatures, DB schema
Time horizon in interview	30-45 min, whole system	30-45 min, one module in depth
Skills tested	Trade-off reasoning, estimation, scale awareness	OOP design, design patterns, clean code
Example question	"Design WhatsApp"	"Design a parking lot / rate limiter class"
Who asks it more	Product companies, senior/staff roles	Product & service companies, all levels

5. Common Interview Questions

Q1: "What is the difference between HLD and LLD?"

English:

Model approach: Define both in one sentence each, then give a concrete example splitting a system into HLD and LLD pieces. Mention that HLD deliverables are diagrams and trade-offs, LLD deliverables are class-level designs. Keep it under 90 seconds — this is usually a warm-up question.

Hindi:

Model approach: दोनों को एक-एक sentence में define करें, फिर एक concrete example दें जिसमें एक system को HLD और LLD हिस्सों में बांटा गया हो। यह बताएं कि HLD के deliverables diagrams और trade-offs होते हैं, जबकि LLD के deliverables class-level designs होते हैं। इसे 90 seconds से कम में रखें — यह आमतौर पर एक warm-up question होता है।

Important Words:

- **Warm-up question** – शुरुआती / वार्म-अप सवाल

Q2: "How would you approach this differently if it were an LLD round?"

English:

Model approach: Say you'd stop drawing boxes for external services and instead pick one core component, then design its classes, interfaces, and design patterns, plus the exact DB schema for that component only.

Hindi:

Model approach: बताएं कि आप बाहरी services के लिए boxes बनाना बंद कर देंगे और इसके बजाय एक core component चुनकर, उसकी classes, interfaces, design patterns design करेंगे, साथ ही सिर्फ उसी component का पूरा DB schema भी।

Q3: "Is this an HLD or LLD question?" (interviewer deliberately ambiguous)

English:

Model approach: This tests whether you ask clarifying questions instead of guessing. The correct move is to ask the interviewer directly: "Should I focus on overall architecture and scale, or on class-level design?" Asking this signals seniority — it is a legitimate, expected question, not a weakness.

Hindi:

Model approach: यह test करता है कि क्या आप guess करने के बजाय clarifying questions पूछते हैं। सही तरीका यह है कि interviewer से सीधे पूछें: "क्या मुझे overall architecture और scale पर focus करना चाहिए, या class-level design पर?" ऐसा पूछना seniority दिखाता है — यह एक सही और expected question है, कोई कमजोरी नहीं।

Important Words:

- **Legitimate** – उचित / सही

Q4: "Give an example of an HLD decision you made in a real project."

English:

Model approach: Pick a real decision from your 9 years of experience, state the context, the alternative you didn't choose, and the concrete consequence you were avoiding. Interviewers weight real, lived trade-off reasoning highly — it's harder to fake than a rehearsed answer.

Hindi:

Model approach: अपने 9 साल के experience में से कोई एक असली decision चुनें, उसका context बताएं, वह alternative जो आपने नहीं चुना, और वह असली consequence जसिसे आप बचना चाहते थे। Interviewers असली, अनुभव से नकिली trade-off वाली सोच को बहुत महत्व देते हैं — इसे बना-बनाया (rehearsed) answer देने से ज़्यादा नकली बनाना मुश्किल होता है।

Important Words:

- **Rehearsed** – पहले से रटा हुआ

6. Practice Exercises

English:

- Take "Design an online food ordering system." Write two separate lists: HLD decisions and LLD decisions. Aim for at least 6 items in each list.
- Sketch an HLD-only box diagram (boxes and arrows, no class details) for a URL shortener, using only what you know today.
- Explain out loud, in under 2 minutes, why a company would run both an HLD interview and a separate LLD interview for the same candidate.

Hindi:

- "Design an online food ordering system" को लें। दो अलग-अलग lists बनाएं: HLD decisions और LLD decisions। हर list में कम से कम 6 items रखने की कोशिश करें।
- एक URL shortener के लिए सिर्फ HLD-level box diagram (boxes और arrows, कोई class details नहीं) बनाएं, सिर्फ वही use करें जो आपको आज पता है।
- ज़ोर से बोलकर, 2 मिनट से कम में समझाएं कि कोई company एक ही candidate के लिए HLD और LLD के अलग-अलग interviews क्यों रखेगी।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"HLD Vs LLD System Design Essentials"	Direct, focused comparison with examples.
English	"High Level Design vs Low Level Design HLD vs LLD System Design Concepts"	Reinforces the distinction.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp (Shradha Khapra)	Frames HLD/LLD inside a full prep roadmap.
Hindi	"HLD vs LLD in System Design Explained Simply!" (Short)	Quick Hindi 60-second refresher.

8. Key Takeaways

English:

- System design is a discipline you already practice informally; HLD makes it deliberate and applies it to a whole product.

- HLD = architecture & component interaction. LLD = internal implementation of one component.
- HLD deliverables are diagrams, API contracts, and capacity estimates -- not code.
- Companies test HLD and LLD in separate rounds because they test different skills.
- If a question is ambiguous about scope, ask the interviewer whether they want HLD or LLD depth.
- This entire program is scoped to HLD; LLD is intentionally out of scope.

Hindi:

- System design वह चीज़ है जो आप पहले से ही informal तरीके से करते हैं; HLD इसे जानबूझकर करता है और पूरे product पर apply करता है।
- HLD = architecture और components का आपस में interaction। LLD = एक component का internal implementation।
- HLD के deliverables diagrams, API contracts, और capacity estimates होते हैं -- code नहीं।
- Companies HLD और LLD को अलग-अलग rounds में test करती हैं क्योंकि दोनों अलग-अलग skills हैं।
- अगर किसी question का scope साफ ना हो, तो interviewer से पूछें कि उन्हें HLD चाहिए या LLD depth।
- यह पूरा program सिर्फ HLD पर focused है; LLD जानबूझकर scope से बाहर रखा गया है।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Ambiguous	अस्पष्ट / अनश्चिति	Not clearly defined, open to interpretation
Architecture	आर्किटेक्चर / ढांचा	The overall structure of a system
Availability	उपलब्धता	How often a system is up and working
Background job	बैकग्राउंड जॉब	A task that runs separately, not blocking the main request
Blueprint	खाका / योजना	A detailed plan for building something
Bottleneck	बॉटलनेक / रुकावट	The part that slows the whole system down
Cache	कैश करना	Storing data temporarily for fast access
Capacity estimate	क्षमता अनुमान	A guess of how much load a system can handle
Checkout	चेकआउट	The process of completing a purchase
Component	कॉम्पोनेंट / हिस्सा	One building block of a system
Cooperating services	साथ मलिकर काम करने वाली services	Services that work together
Deliberately	जानबूझकर	On purpose, with intention
Deliverable	डिलीवरेबल / परिणाम	A concrete output produced from work
Design pattern	डिज़ाइन पैटर्न	A reusable solution to a common coding problem
Inline	इनलाइन / सीधे	Done immediately within the same flow
Instinct	सहज प्रवृत्ति	A natural, automatic reaction
Justification	औचित्य / कारण देना	Explaining why something is the right choice
Latency	लेटेसी / देरी	The time delay in a response
Legitimate	उचित / सही	Valid and acceptable
Muscle (figurative)	क्षमता / skill	A trained ability, used figuratively here
Non-functional requirements	नॉन-फंक्शनल रक्वायरमेंट्स	Quality needs like speed and reliability, not features
Own (verb, figurative)	ज़िम्मेदारी लेना	To take full responsibility for something
Payment gateway	पेमेंट गेटवे	A service that processes online payments

English Word	Hindi Meaning	Simple Explanation
Persistence	परसिस्टेंस / स्थायी भंडारण	Saving data so it survives after the program stops
Proxy	प्रॉक्सी / प्रतिनिधि भाष	A stand-in measure used to test something indirectly
Pseudo-code	स्यूडो-कोड	Code-like notes that describe logic without real syntax
Rehearsed	पहले से रटा हुआ	Practiced and memorized in advance, not spontaneous
Relational database	रलेशनल डेटाबेस	A database organized into related tables
Repository pattern	रिपॉजिटरी पैटर्न	A design pattern for organizing data access code
State machine	स्टेट मशीन	A model where something moves through defined stages
Synchronously	सक्रोनसली / एक साथ क्रम में	Happening in order, one waiting for the other
Validation	वैलिडेशन / जांच	Checking that data is correct before using it
Walkthrough	वॉकथ्रू / वसितार से समझाना	Explaining something step by step
Warm-up question	शुरुआती / वार्म-अप सवाल	An easy first question to start the interview
Workable	व्यावहारिक / काम करने लायक	Practical and functional
Zoom into	अंदर जाकर देखना	Look at something in more detail